

The Run Report, in Plain English

A companion to RUN_REPORT.md — *what actually happened during the build, what each headline number means in practice, and why the deviations and to-dos matter. Written for a non-technical reader; the full glossary lives in the paper companion.*

What the run report is

The run report is the project's "flight log": what was executed, what came out, where reality forced a detour from the original plan, and what remains for the project owner to do by hand. Everything in it was produced by code that ran start-to-finish on the real dataset — and then double-checked: an audit script re-opened every saved result file and confirmed that all **71 numbers quoted in the write-up match their sources exactly**. When the report says "18/18 tests green," it means the project's self-checks — the ones that catch time-machine features and mis-drawn data splits — all passed on the real data, not just in rehearsal.

What each completed stage actually did

Stage 0 — the data gate. Before any analysis, the pipeline proved it had the *right* data: the archive's SHA-256 fingerprint (a 64-character code that changes if even one byte changes) matched the one computed on the original download, and the table's shape matched the official documentation — 590,540 transactions across 182 days, 3.5% fraud. Fail here and everything downstream stops.

Stage 1 — EDA and the leakage audit. A guided look at the data (volumes, gaps, drift over time) plus a written case for why this dataset must never be evaluated with shuffled splits — the "no time machines" rules the rest of the project obeys.

Stage 2 — baselines and the optimism gap. Train the simple model and the powerful one, grade both the popular-but-wrong way and the honest way, and measure the difference. This produced the project's cautionary headline (details below).

Stage 3 — calibration. Repair the model's scores so "30%" means a real 30% chance. The staircase-style fix (isotonic) won the tryout against the smooth-curve fix (Platt).

Stage 4 — the cost layer. Convert probabilities into approve/decline decisions priced in dollars, and compare six decision policies on identical data.

Stage 5 — drift and retraining. Watch performance decay week by week, and measure how much of it monthly retraining buys back.

Stage 6 — ablation. Strip feature groups in and out to see which data earns its complexity.

Stage 7 — explainability. Produce itemized receipts (SHAP) for individual decisions, including one correct catch and one false alarm.

Stage 8 — the write-up and the demo app. The paper you have, plus an interactive app that runs from cached results (no dataset needed).

Stage 9 — investigation narratives. Plain-language case notes for flagged transactions. Generated from fixed templates this run (no AI-service key was configured); the AI pathway ships in the code but deliberately sits outside the real-time decision.

The headline numbers, translated

"Savings at $k = 0.15$: 39.4%, CI [35.8%, 42.8%]." Set the scene: on the never-touched final month, doing nothing (approving everyone) loses **\$495,244** to fraud. The system's decisions cut the month's total damage — fraud still slipping through, plus the cost of good customers wrongly declined — to **\$300,271**. It erased 39.4% of the erasable loss. The bracket is the wobble range: rerun the same month with a slightly different mix of customers and you'd expect somewhere between 35.8% and 42.8%. The " $k = 0.15$ " is the price assumption behind it: a wrongly declined customer hurts 15% as much as a missed fraud of the same size. Change that assumption and the system adapts (the paper stress-tests k from 0.05 to 1.0).

"TPR @ 5% FPR: 67.2%." Cap the false-alarm rate at 5 in 100 legitimate transactions — an industry-style operating point — and the system catches **just over two-thirds of all fraud attempts**. The two numbers move together: catch more, annoy more. This metric fixes the annoyance and reports the catch.

"Optimism gap: random CV overstates PR-AUC by 0.30." The same model, graded two ways. The shuffled-data method (still common in tutorials and competitions) reported a precision score of 0.87; the honest future-month test reported 0.56. Nothing about the model changed — only the grading. Practically: *a team using the shuffled method would have promised roughly a third more precision than production would deliver*. The simple benchmark model barely inflated at all, which pins the blame on memorization of repeat card behavior, not on bad luck.

"Calibration effect: +2.2 points of savings = \$11,132 on the month." Feeding the decision rule raw scores instead of calibrated ones would have cost about eleven thousand dollars extra in one month on this portfolio. The report is careful here: the month-to-month wobble range on this *particular* difference slightly crosses zero ($[-0.2, +4.3]$), so it's reported as "consistently positive (96% of re-samples) but not ironclad on a single month" — an honesty standard worth noticing. The ironclad result is the next one.

"Analytic threshold beats F1 by +7.9 points, CI [+5.1, +10.8]." F1 is a popular accuracy formula for choosing the decline bar; it knows nothing about money. The cost-aware bar (decline above 13% probability, given $k = 0.15$) beat it by 7.9 points of savings, positive in 100% of a

thousand re-samples. And the other accuracy-formula bar (Youden's J) doesn't just underperform when false declines get pricier — at $k = 1.0$ it destroys value (−238%), because a threshold chosen without reference to costs keeps declining aggressively as declining gets more expensive. *Where you set the bar matters more than how clever the model is — and the right bar is a one-line formula once scores are calibrated.*

"Best cadence: monthly expanding retraining, +7.5 points over static." A model trained once and left alone loses about 7.5 points of savings by month 5 compared to one refreshed monthly with all accumulated history. Retraining on only a recent window was slightly worse than keeping everything — *freshness matters more than forgetting.*

"Ablation: 40 features carry 86% of the value." Most of the money comes from a small, explainable core (basic transaction facts plus simple counting signals). The hundreds of opaque extra signals add real but modest value. For a bank's model-governance office, that's the difference between a system you can document and one you can't.

"Adversarial validation AUC 0.964; top PSI: the browser-version field." A classifier could easily tell "training-period data" from "later data" — proof the world shifted under the model. The single most-shifted input was the customer's browser version: software updates alone quietly age a fraud model.

The deviations, honestly

1. **The dataset arrived by hand.** The build environment had no route to Kaggle (or to any package repository), so the owner downloaded the official archive and supplied it through a folder bridge — in 60 MB slices, because the bridge silently dropped anything bigger. The reassembled file's fingerprint matched the original exactly, so provenance is intact. Rejected alternative: downloading a copy from an unofficial mirror (unverifiable and against the competition's terms).
2. **A lighter hyperparameter search (12 trials).** The build machine had 2 processor cores; the tuning stage was capped at its 45-minute budget rather than allowed to sprawl. The searching *method* stayed exactly as specified. Deeper tuning might add a little accuracy; it wouldn't change any conclusion, since every comparison holds the tuning fixed.
3. **XGBoost omitted.** The plan allowed either of two gradient-boosting libraries; the specified primary (LightGBM) was used. The alternate library's 132 MB installer wouldn't survive the file bridge, and it would have been redundant anyway.
4. **Two citations dropped.** Two statistics suggested for the introduction could not be verified against their supposed sources at build time — one report turned out not to contain the quoted figures, one paper couldn't be found at all. Both were cut and the substitutions documented. (The same discipline applied to the project's own numbers: all 71 machine-verified.)

5. **Template narratives.** The AI-written case notes ran in fallback mode because no AI-service key was present; 400 template-based notes were generated instead, and the design keeps any AI strictly out of the real-time decision anyway.
6. **Runtime 4¾ hours, not 2-3.** Same pipeline, half the usual processor cores; per-stage timings are documented and a normal laptop lands in the target range.

What's left for the owner (and why)

1. **Put your surname in the paper and license.** Placeholders were left because documents that ship with someone's name should be placed there by that someone.
2. **Push to GitHub.** The repository arrives with clean history; publishing it is a one-command decision that makes the work citable on a resume.
3. **Decide how to host the demo.** The interactive app runs from a cached sample of the competition's data. The competition's rules restrict redistributing data — so the honest options are: run it locally, host it privately, or swap in synthetic rows before any public deployment. That decision belongs to the owner, not to an automated pipeline.
4. **Write the resume bullet from the real numbers.** A suggested one is included in the run report; every figure in it traces to an audited result file — which is precisely the property that makes it safe to say in an interview.
5. **Skim the decision log (DECISIONS.md).** Eleven dated entries record every judgment call and every rejected alternative. In an interview, "here's the log of every decision I made and why" is itself a differentiator.

Every number quoted here comes from the same audited result files as the paper; nothing in this companion is a new claim.